

Sprawozdanie techniczne

Space Shooter 2D

Itch.io: <https://bartosz-kozłowski.itch.io/space-shooter2d>

YouTube: <https://www.youtube.com/watch?v=G1TrgyDD0s>

1. Wprowadzenie

Niniejszy dokument opisuje architekturę gry Space Shooter 2D zrealizowanej w silniku Unity jako dwuwymiarowa strzelanka z widokiem pionowym. Gracz steruje statkiem kosmicznym, niszczy nadlatujące fale przeciwników, zbiera ulepszenia (power-upy) oraz wydaje zdobyte punkty na trwałą progresję statku w sklepie ulepszeń.

2. Ogólna architektura projektu

Gra zbudowana jest wokół czterech scen, które tworzą pełny cykl rozgrywki:

- MainMenu - menu główne z przyciskiem rozpoczęcia gry oraz ustawieniami głośności.
- CharSelectScene - ekran wyboru jednego z dwóch statków o różnych parametrach.
- GameScene - właściwa rozgrywka: spawning wrogów, strzelanie, power-upy, HUD oraz sklepik ulepszeń.
- GameOver - ekran końca gry prezentujący wynik końcowy z możliwością restartu.

Logika podzielona jest na wyspecjalizowane komponenty, z których każdy odpowiada za jeden obszar działania. Obiekty współdzielone między scenami (menedżer dźwięku, licznik punktów, wybór postaci) zaimplementowano jako Singletons przeżywające zmianę sceny dzięki mechanizmowi DontDestroyOnLoad. Pozostałe menedżery (np. UpgradeManager, EnemySpawner) tworzone są na nowo wraz z każdym załadowaniem GameScene, co gwarantuje czysty stan przy starcie każdej rozgrywki.

2.1. Kluczowe komponenty

- PlayerController - ruch gracza i maszyna stanów strzelania (zwykły ogień / ładowanie / strzał naładowany).
- Shooter - tworzenie pocisków, obsługa AI wrogów oraz efektu MultiShot.
- Health - zarządzanie zdrowiem gracza i wrogów, obsługa obrażeń, tarczy i śmierci.
- EnemySpawner - spawning fal przeciwników według konfiguracji ScriptableObject.
- PathFinding - ruch wrogów po wyznaczonej trasie waypointów.
- PowerUpSpawner / PowerUpManager - losowe upuszczanie i zbieranie wzmocnień.
- UpgradeManager / UpgradeMenuController - system progresji statku (sklepik).
- AudioManager / MusicController - efekty dźwiękowe, muzyka i kontrola głośności.
- LevelManager / ScoreKeeper / CharSelectManager - przejścia między scenami, wynik i wybór statku.

3. Pełny Game Loop

Gra realizuje kompletny cykl rozgrywki od menu startowego, przez rozgrywkę, aż po ekran końcowy z możliwością ponownego uruchomienia.

Przebieg wygląda następująco. Po uruchomieniu gracz trafia do menu głównego, skąd rozpoczyna grę i przechodzi do ekranu wyboru statku. Wybiera jedną z dwóch postaci (różniących się parametrami - wytrzymałym, lecz wolniejszym statkiem niebieskim oraz szybkim, lecz delikatnym pomarańczowym) i rozpoczyna właściwą rozgrywkę. W trakcie gry steruje statkiem, unikając nadlatujących przeciwników i ich pocisków oraz ostrzeliwując wrogów - strzałem zwykłym lub mocniejszym strzałem naładowanym. Za pokonanych przeciwników gracz zdobywa punkty, a z części z nich wypadają power-upy (np. MultiShot, tarcza, leczenie), które można zebrać, przelatując przez nie statkiem. Zebrane punkty można na bieżąco wymieniać na trwałe ulepszenia w sklepiu.

Gdy zdrowie gracza spadnie do zera, statek zostaje zniszczony i następuje przejście do ekranu Game Over prezentującego wynik końcowy. Z tego ekranu gracz może rozpocząć rozgrywkę od nowa lub wrócić do menu głównego - cykl zamyka się i może rozpocząć się ponownie od czystego stanu.

Przejścia między scenami sterowane są centralnie przez klasę LevelManager.

Przy powrocie do menu wybór statku jest resetowany, a przy starcie nowej gry zerowany jest wynik - dzięki temu każda kolejna rozgrywka zaczyna się od czystego stanu.

4. Zarządzanie scenami

Do ładowania i przeładowywania poziomów wykorzystano przestrzeń nazw UnityEngine.SceneManagement oraz metodę SceneManager.LoadScene. Wszystkie przejścia skupione są w jednej klasie LevelManager, co centralizuje logikę nawigacji i ułatwia jej utrzymanie.

Część obiektów musi zachować swój stan pomiędzy scenami - np. wybrany statek (potrzebny w GameScene po dokonaniu wyboru w CharSelectScene) oraz zdobyty wynik (przenoszony do ekranu GameOver). Zrealizowano to wzorcem Singleton w połączeniu z DontDestroyOnLoad.

Opóźnione przejście do innej sceny realizowane jest przy pomocy coroutine (WaitAndLoad), co pokazuje wykorzystanie mechanizmu współprogramów Unity do sterowania czasem.

5. Zastosowane wzorce projektowe

5.1. Singleton

Wzorec Singleton zapewnia istnienie dokładnie jednej instancji obiektu i globalny dostęp do niej. Zastosowano go dla klas, które muszą być unikalne i dostępne z dowolnego miejsca: AudioManager, ScoreKeeper oraz CharSelectManager. Każda z tych klas chroni się przed powstaniem duplikatu i przeżywa zmianę sceny.

5.2. Observer (zdarzenia)

Wzorec Obserwatora realizowany jest przez zdarzenia języka C# (event Action). Obiekt-nadawca powiadamia subskrybentów o zmianie stanu, nie wiedząc nic o ich konkretnej implementacji. W projekcie wykorzystano go w dwóch miejscach.

Pierwszy przykład: ChargeBarManager zgłasza moment pełnego naładowania paska, a PlayerController na to reaguje.

PlayerController subskrybuje to zdarzenie w Awake i odsubskrybowuje w OnDestroy, co zapobiega wyciekom pamięci.

Drugi przykład: UpgradeManager zgłasza zdarzenie OnUpgradeChanged po każdym zakupie ulepszenia, na które reaguje warstwa interfejsu (UpgradeMenuController), odświeżając etykiety i kolory przycisków. Dzięki temu logika gry jest całkowicie oddzielona od warstwy UI.

5.3. State Pattern (maszyna stanów)

Logika strzelania gracza zorganizowana jest jako maszyna stanów. W zależności od aktualnego stanu (oczekiwanie na naładowanie, ładowanie w toku, gotowy strzał naładowany, zwykły ogień) system zachowuje się inaczej. Stan oceniany jest co klatkę w metodzie FireShooter.

5.4. Factory (fabryka)

Tworzenie statku gracza zamknięte jest w klasie PlayerInstantiate pełniącej rolę fabryki. Reszta kodu nie zna prefabów ani szczegółów tworzenia - jedynie deleguje decyzję na podstawie indeksu wybranego statku.

5.5. Strategy (strategia)

Power-upy zaprojektowano wzorcem Strategii opartym o ScriptableObject. Abstrakcyjna klasa bazowa PowerUpSO definiuje wspólny kontrakt (metodę Apply), a każdy konkretny typ power-upa dostarcza własną implementację efektu. Kod zbierający power-up wywołuje Apply nie wiedząc, jaki to konkretnie typ.

6. Fizyka i interakcje

Projekt opiera się na fizyce 2D silnika Unity. Pociski oraz power-upy posiadają komponent Rigidbody2D, któremu nadawana jest prędkość liniowa (linearVelocity), dzięki czemu silnik fizyki samodzielnie zarządza ich ruchem.

Wykrywanie kolizji zrealizowano przy pomocy colliderów w trybie wyzwalacza (Is Trigger) i metody OnTriggerEnter2D. Komponent Health wykrywa trafienie pociskiem, pobiera wartość obrażeń, odtwarza efekt cząstek i dźwięk, a następnie niszczy pocisk.

Analogicznie działa zbieranie power-upów: komponent PowerUpManager w metodzie OnTriggerEnter2D weryfikuje (poprzez warstwę Layer oraz obecność odpowiednich komponentów), czy obiekt dotykający to gracz, i dopiero wtedy aplikuje efekt. Statki posiadają także collidery (CapsuleCollider2D), a tarcza ochronna - włączany dynamicznie CircleCollider2D.

7. Instancjonowanie obiektów

Dynamiczne tworzenie obiektów w czasie gry odbywa się metodą Instantiate i występuje w wielu miejscach systemu:

- Pociski - tworzone przez Shooter przy każdym strzale (zwykłym, naładowanym oraz w trybie MultiShot pod różnymi kątami).

- Przeciwnicy - spawnowani przez EnemySpawner w pętli coroutine na podstawie konfiguracji fal.
- Power-upy - upuszczane przez PowerUpSpawner po śmierci wroga z określonym prawdopodobieństwem.
- Statek gracza - tworzony przez PlayerInstantiate na początku GameScene.
- Efekty cząstek - instancjonowane przy każdym trafieniu i automatycznie niszczone po zakończeniu animacji.

Spawning przeciwników zrealizowano w postaci współprogramu, który iteruje po falach i pojedynczych wrogach, zachowując odstępy czasowe.

Obiekty o ograniczonym czasie życia (pociski, power-upy, cząstki) są automatycznie usuwane przez Destroy z opóźnieniem, co zapobiega niekontrolowanemu przyrostowi obiektów na scenie.

8. ScriptableObjects

Do przechowywania danych i parametrów gry zastosowano obiekty ScriptableObject, które oddzielają dane od logiki i pozwalają na ich edycję bezpośrednio w edytorze Unity, bez ingerencji w kod.

WaveConfigSO opisuje pojedynczą falę przeciwników: tablicę prefabów wrogów, trasę poruszania (waypoints), prędkość, czas między spawnami oraz jego losową wariację. EnemySpawner i PathFinding czytają te dane w czasie rozgrywki, dzięki czemu układ fal można projektować całkowicie z poziomu edytora.

Hierarchia PowerUpSO (z konkretnymi typami HealthPowerUp, ShieldPowerUp, MultiShotPowerUp) przechowuje zarówno dane (np. ilość leczenia, czas trwania efektu, liczba dodatkowych pocisków), jak i - dzięki wzorcowi Strategii - samą logikę zastosowania efektu. Taka konstrukcja pozwala dodawać nowe rodzaje wzmocnień bez modyfikacji istniejącego kodu.

9. UI / UX

Interfejs użytkownika obejmuje kompletne menu główne oraz rozbudowany HUD informujący o stanie rozgrywki.

- Menu główne (MainMenu) - przycisk rozpoczęcia gry oraz panel ustawień głośności z osobnymi suwakami Master, Music i SFX.
- HUD - pasek zdrowia gracza, licznik punktów oraz animowany pasek ładowania strzału naładowanego.
- Sklepik ulepszeń - panel z trzema kategoriami ulepszeń, kosztami i poziomami, dostępny w trakcie gry.
- Ekran GameOver - prezentacja wyniku końcowego z opcją restartu.

HUD aktualizowany jest na bieżąco przez klasę UIUpdater, która co klatkę odświeża wynik oraz wypełnienie paska zdrowia (czytając wartości dynamicznie, aby pasek pozostał poprawny również po zwiększeniu maksymalnego zdrowia ulepszeniem). Kolorystyka interfejsu dostosowuje się do wybranego statku. Otwarcie dowolnego menu w trakcie gry zatrzymuje rozgrywkę poprzez ustawienie Time.timeScale na zero oraz blokuje strzelanie, co zapobiega akcjom w tle podczas przeglądania menu.

10. Multimedia (dźwięk i animacje)

Warstwa audio oparta jest o AudioManager z trzema grupami głośności: Master, Music i SFX. Centralny AudioManager (Singleton) odtwarza efekty dźwiękowe metodą PlayOneShot oraz steruje głośnością poszczególnych grup. Konwersja wartości suwaka na decybele odbywa się logarytmicznie, zgodnie z charakterystyką ludzkiego słuchu, a ustawienia są trwale zapamiętywane między uruchomieniami gry przy pomocy PlayerPrefs.

W grze występują efekty dźwiękowe strzału, otrzymania obrażeń, ładowania oraz wystrzelenia strzału naładowanego, a także podkładowa muzyka odtwarzana w pętli z osobnego źródła podpiętego do grupy Music.

Animacje zrealizowano przy pomocy komponentu Animator i kontrolerów animacji. Stany animacji statku (ładowanie, strzał naładowany) przełączane są z poziomu kodu poprzez parametry kontrolera (metoda SetBool), co oddziela logikę gry od warstwy prezentacji wizualnej. Animowane są między innymi: statek gracza, efekty strzałów, tarcza ochronna oraz przeciwnicy. Dodatkowo zaimplementowano efekt drżenia kamery (CameraShake) przy trafieniu, zwiększający odczucie siły uderzenia.

11. Autorska mechanika rozszerzona - system progresji (sklepik)

Funkcja wyróżniająca projekt (Plus Feature) to system progresji statku w postaci sklepiku ulepszeń dostępnego w trakcie rozgrywki. Punkty zdobywane za niszczenie przeciwników pełnią jednocześnie rolę wyniku oraz waluty, którą gracz może wymieniać na trwałe wzmocnienia.

Dostępne są trzy kategorie ulepszeń, każda w pięciu poziomach o rosnącym koszcie:

- DAMAGE - zwiększa obrażenia zadawane przez pociski.
- RELOAD - skraca czas między strzałami oraz czas ładowania strzału naładowanego.
- HEALTH - zwiększa maksymalne zdrowie statku.

Logikę progresji obsługuje UpgradeManager, który weryfikuje środki, pobiera punkty (operacja atomowa - albo zakup dochodzi do skutku w całości, albo wcale) i nalicza efekt ulepszenia. Koszt kolejnego poziomu rośnie liniowo, a po osiągnięciu maksymalnego poziomu dana kategoria zostaje zablokowana.

Efekty ulepszeń aplikowane są w różnych miejscach systemu: mnożnik obrażeń nakładany jest na pocisk w momencie jego tworzenia (Shooter), mnożnik przeładowania skraca czas między strzałami i czas ładowania (Shooter oraz ChargeBarManager), a ulepszenie zdrowia natychmiast zwiększa maksymalne i bieżące zdrowie gracza (Health). Po każdym zakupie UpgradeManager zgłasza zdarzenie OnUpgradeChanged, dzięki któremu interfejs sklepiku automatycznie aktualizuje koszty, poziom oraz dostępność przycisków.

System progresji łączy ze sobą wiele mechanik gry (punktację, strzelanie, zdrowie, interfejs) i stanowi spójne rozszerzenie podstawowej rozgrywki, nagradzając gracza za skuteczną eliminację przeciwników i wprowadzając element decyzji strategicznych dotyczących rozwoju statku.